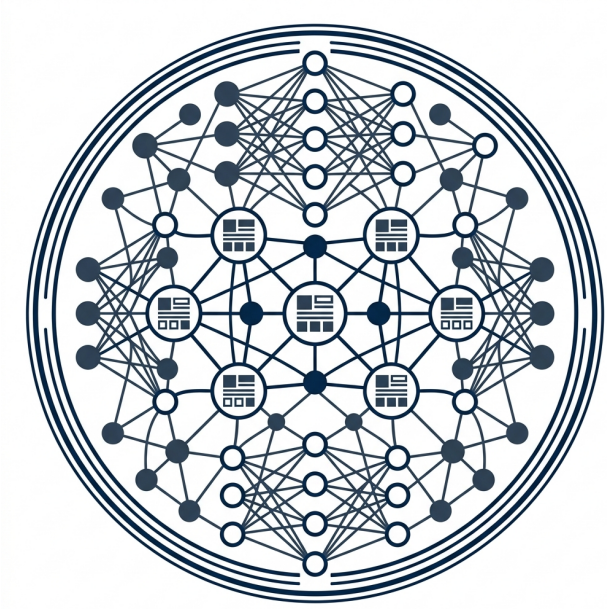
PROYECTO FINAL DE MINERÍA DE TEXTOS

WikiRAG-ES

Asistente RAG local sobre Wikipedia en español

Arquitectura agencial, recuperación híbrida y evaluación IR



AUTORES

Lucas González Fiz
Adan González Rodríguez
Santiago Palomino Vazquez
Javier Pérez Robles

Índice

1. Resumen del Proyecto **1**

2. Requisitos técnicos **1**
2.1. Módulo para cada temática . . . 1
2.2. Archivos entregados . . . 2

3. Entorno de Ejecución **2**
3.1. Tecnologías y librerías clave . . . 2
3.2. Hardware utilizado en pruebas . . . 2

4. Arquitectura del Sistema **3**
4.1. Diagrama de Flujo de Datos . . . 3
4.2. Diseño Agencial del Router . . . 3
4.3. Resumen extractivo y determinismo . . . 4

5. Corpus de Conocimiento (MIRACL-es) **5**
5.1. Adquisición, Limpieza y Preprocesado . . . 5
5.2. Revisión del modelado de tópicos . . . 5

6. Resultados de Evaluación IR **6**
6.1. Métricas obtenidas . . . 6
6.2. Qué mide cada métrica . . . 6
6.3. Análisis: del baseline al sistema completo . . . 7
6.4. Impacto del Reranking (Cross-Encoder) . . . 7

7. Escalabilidad RAG según el Tamaño del Corpus **8**

8. Indexación Aproximada (HNSW) **9**

9. Benchmark de Cuantización (TurboVec) **10**

10. Comparativa de Técnicas Semánticas **10**

11. Ejemplos Reales del Asistente RAG **11**
11.1. Consulta Factual (QREL Experto) . . . 11
11.2. Rechazo Informado (Prevención de Alucinaciones) . . . 12
11.3. Llamada Agencial (Tool Routing) . . . 12

12. Discusión y Limitaciones Conocidas **13**

13. Conclusiones **13**

A. Apéndice: Manual de Reproducción y Ejecución **14**

1. Resumen del Proyecto

Se ha diseñado e implementado un **asistente documental local** basado en la arquitectura de Recuperación Aumentada por Generación (*Retrieval-Augmented Generation*, RAG). El sistema descarga, normaliza y procesa un corpus masivo real (Wikipedia en español), fragmentándolo en *chunks* y generando representaciones vectoriales densas. A partir de estos datos, construye índices híbridos (FAISS y BM25) y ejecuta modelado de tópicos *offline*, permitiendo a un Agente LLM enrutar y responder consultas complejas en lenguaje natural.

- **Corpus:** 1 000 000 documentos de MIRACL-es (Wikipedia en español), con 200 consultas *qrels* de evaluación.
- **Escala de indexación:** 1 140 063 *chunks* procesados e indexados.
- **Motor LLM:** `gemma4:e4b` operando íntegramente en local a través de Ollama.
- **Recuperación:** Sistema híbrido que fusiona recuperación semántica (FAISS) y léxica (BM25) mediante *Reciprocal Rank Fusion* (RRF), con *reranking* opcional por *cross-encoder*.
- **Eficiencia:** Carga parcial de índices por modo de evaluación, ruta rápida para herramientas deterministas y cuantización vectorial TurboVec que reduce la memoria utilizada por los vectores en un 93,4 %.

2. Requisitos técnicos

2.1. Módulo para cada temática

Esta es la lista de los requisitos técnicos del proyecto.

Requisito	Módulo	Descripción
Embeddings	`indexing/embeddings.py`	Modelo Sentence Transformers multilingüe (384-dim).
Modelado de tópicos	`topics/modeling.py`	Agrupación semántica *offline* con BERTopic/NMF.
Resumen extractivo	`summarization/extractive.py`	TextRank sobre grafos + MMR para penalizar redundancia.
LLM Local	`llm/ollama.py`	Conexión nativa con Ollama, hiperparámetros ajustados.
Pipeline RAG	`retrieval/hybrid.py`	Fusión híbrida FAISS + BM25 mediante RRF, independiente de escalas de *score*.
Router Agencial	`agents/router.py`	Enrutado híbrido: atajos deterministas por regex y desambiguación con el LLM (salida JSON).
Innovación	`indexing/turbovec_store.py`	Compresión extrema de vectores mediante cuantización por anchura de bits (TurboQuant).

Cuadro 1: Mapeo de requisitos del enunciado y módulos desarrollados.

2.2. Archivos entregados

El proyecto se acompaña de los siguientes archivos:

Ruta del archivo o directorio	Contenido y función
`src/wikirag_es/`	Código fuente completo de la librería Python del paquete `wikirag-es`.
`data/processed/miracl_es_corpus_1M.jsonl`	Corpus local de 1 000 000 documentos MIRACL-es.
`data/reports/miracl_es_eval.jsonl`	200 consultas de evaluación con sus documentos relevantes disponibles dentro del subconjunto local.
`data/indexes/default/`	Índices FAISS (Flat) y BM25 generados sobre 1 140 063 fragmentos.
`data/reports/doc_eval_*.json`	Reportes de evaluación IR reproducidos para 50 consultas etiquetadas.
`data/reports/*bench*.json`	Resultados y métricas de los benchmarks empíricos de escalabilidad, latencia y compresión de memoria (HNSW, TurboVec).
`data/reports/miracl_es_corpus_package.tar.gz`	Entregable comprimido del corpus local, consultas de evaluación y manifiesto. El informe de tópicos se entrega como `data/reports/topics.json`.

Cuadro 2: Estructura de entregables del sistema.

3. Entorno de Ejecución

3.1. Tecnologías y librerías clave

Capa de software	Tecnología
Entorno y CLI	Python 3.12, Click. El entorno de prueba actual tiene Click 8.3.0.
Modelado Vectorial	Sentence Transformers, modelo `paraphrase-multilingual-MiniLM-L12-v2`. El entorno de prueba actual tiene Sentence Transformers 5.4.1 y PyTorch 2.11.0+cu128.
Bases de Datos Vectoriales	FAISS CPU (IndexFlatIP / IndexHNSWFlat) en el entorno de benchmark original. TurboVec como dependencia opcional para cuantización a 4 bits.
Recuperación Léxica y *Reranking*	BM25Okapi (`rank-bm25`) y cross-encoder `mmarco-mMiniLMv2-L12`
Motor de Inferencia	Ollama (servidor local), modelo `gemma4:e4b`
Procesamiento Analítico	BERTopic (UMAP+HDBSCAN), NetworkX (PageRank/TextRank)

Cuadro 3: Stack tecnológico del proyecto.

3.2. Hardware utilizado en pruebas

Para acelerar la generación de *embeddings* de más de un millón de fragmentos se ha aprovechado aceleración por hardware (CUDA).

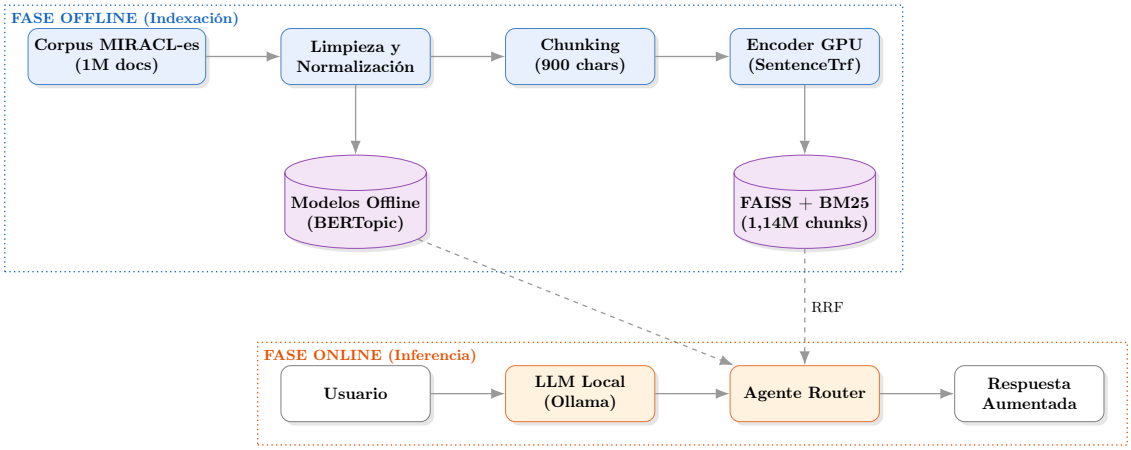
Componente	Especificación
Acelerador Gráfico (GPU)	NVIDIA GeForce RTX 5060 Ti (8 GB VRAM)
Motor Vectorial	PyTorch con backend CUDA en la corrida de indexación. En la prueba actual se detectó PyTorch 2.11.0+cu128, mientras que `faiss-cpu`, `bertopic` y `turbovec` no estaban instalados en el intérprete activo.

Cuadro 4: Especificaciones del servidor local utilizado en la corrida documentada.

4. Arquitectura del Sistema

El flujo de trabajo se divide conceptualmente en dos grandes bloques: el procesamiento de datos por lotes (fase *offline*) y la interacción con el LLM en tiempo real (fase *online*).

4.1. Diagrama de Flujo de Datos



4.2. Diseño Agencial del Router

En lugar de construir un RAG tradicional donde toda pregunta activa incondicionalmente la base vectorial, se ha implementado un **Agente Router** de dos niveles. La clase `AssistantAgent` aplica primero atajos deterministas por expresiones regulares para disparadores explícitos (resumen, fecha, tópicos, peticiones web y consultas de actualidad) y, ante consultas ambiguas, delega la decisión en el **LLM**, que selecciona la herramienta devolviendo un objeto JSON (`function-calling` ligero). Si el LLM no está disponible o su salida es inválida, el sistema recae con seguridad en la herramienta RAG. Esto ahorra tokens, reduce la latencia previa a mostrar la herramienta activa y mantiene un comportamiento robusto. El agente conserva además **memoria conversacional**: solo reescribe las preguntas de seguimiento cuando hay referencias anafóricas reales al historial.

Herramienta	Patrón de activación	Ejecución algorítmica
`RagTool`	(Fallback por defecto)	Ejecuta FAISS y BM25, fusiona con RRF e inyecta fragmentos al LLM.
`SummaryTool`	*resume, ideas principales...*	Calcula TextRank extractivo sobre el contexto recuperado y devuelve frases existentes.
`TopicTool`	*tópicos/clusters del corpus...*	Extrae información del archivo JSON de clústeres de BERTopic.
`DateTool`	*qué día es hoy...*	Devuelve la fecha actual formateada en español (día de la semana incluido).
`WebSearchTool`	*internet, web, noticias, actualidad...*	Busca en DuckDuckGo información complementaria, la combina con contexto local y responde citando `[Web N]` cuando usa resultados externos.

Cuadro 5: Inventario de herramientas del Agente Router.

Las respuestas generativas se emiten en *streaming* mediante Ollama cuando pasan por el LLM. Las rutas deterministas sin LLM, como fecha y resumen extractivo, emiten texto por bloques o frases para que el usuario no espere a que termine toda la operación. Además, las fuentes mostradas al final de una respuesta RAG o web se filtran por citas explícitas: solo se listan aquellas que aparezcan realmente como `[Fuente N]` o `[Web N]` en la respuesta. Esto evita presentar como evidencias documentos recuperados por vecindad cuando el modelo ha declarado que no hay contexto suficiente.

4.3. Resumen extractivo y determinismo

El módulo de resumen no es un resumen abstractivo generado por el LLM. Es un **resumen extractivo**: selecciona frases literales presentes en el contexto recuperado o en el fichero de entrada. La implementación se encuentra en `summarization/extractive.py` y combina tres pasos:

1. Segmentación del texto en frases suficientemente largas.
2. Cálculo de embeddings por frase y construcción de un grafo de similitud coseno.
3. Aplicación de TextRank/PageRank para centralidad y selección con MMR para penalizar redundancia.

Cuando el usuario pregunta *"Resume lo que tengas sobre X"*, el agente limpia la consulta para recuperar por el tema principal, obtiene contexto con RAG y después resume ese contexto. Antes de aplicar TextRank se eliminan cabeceras de contexto como `[Fuente N]`, `Ruta:` y `Texto:`; el resumen se calcula sobre el contenido documental, no sobre metadatos. Por tanto, el resumen es tan estable como lo sean la consulta efectiva y el contexto recuperado. Con el mismo índice, mismo `top_k`, mismo historial conversacional y mismo contexto de entrada, el algoritmo extractivo debe devolver el mismo resultado. Si cambia el historial, se activa un índice aproximado, cambia el reranker o se recuperan fragmentos distintos, el resumen también puede cambiar.

La revisión del sistema detectó tres fuentes de variabilidad que debían controlarse: la intención explícita de resumen podía perderse si la reescritura conversacional eliminaba la palabra *"resume"*, la consulta de resumen podía incluir verbos de instrucción que deterioraban el recuperador, y la selección MMR podía desempatar frases equivalentes sin orden estable. La corrección aplicada hace que los disparadores explícitos del usuario tengan prioridad sobre la reescritura, que la búsqueda se haga por el tema limpio y que los empates del resumen se resuelvan por orden de frase. Esto no convierte el RAG completo en determinista bajo cualquier configuración, pero sí hace determinista el resumen cuando recibe el mismo texto de entrada.

5. Corpus de Conocimiento (MIRACL-es)

Se eligió MIRACL (*Multilingual Information Retrieval Across a Continuum of Languages*) como corpus principal por tres motivos fundamentales:

1. Contiene volcados directos de Wikipedia, asegurando alta densidad informativa.
2. Garantiza gramática y vocabulario nativo en español.
3. Proporciona conjuntos de evaluación manual (*qrels*) validados por expertos, indispensables para poder medir objetivamente el rendimiento del sistema RAG.

Métrica del Corpus	**Volumen final**
Documentos indexados	1 000 000 documentos MIRACL-es
Chunks resultantes indexados	1 140 063 fragmentos de texto
Consultas de evaluación (*qrels* expertos)	200
Tamaño persistido (FAISS + `embeddings.npy`)	3,50 GB de vectores
Tópicos offline entregados	BERTopic sobre 100 198 documentos, 1 413 filas de `topic_info`

Cuadro 6: Estadísticas del corpus de entrega tras la fase de ingestión.

5.1. Adquisición, Limpieza y Preprocesado

Antes de indexar los documentos en la base vectorial, se aplicaron varios pasos de limpieza y preparación de los textos para asegurar que la información fuera legible y útil:

- **Adquisición de datos:** El módulo `miracl.py` descarga y lee los datos en *streaming*, lo que evita saturar la memoria RAM al procesar el gran volumen del dataset original.
- **Limpieza de texto:** En `utils/text.py` se usa la librería `ftfy` para arreglar automáticamente errores de codificación (por ejemplo, ver "Ã¡" en lugar de "á") y se eliminan caracteres invisibles que puedan dar problemas.
- **Filtro de seguridad:** Se aplican expresiones regulares (`strip_prompt_injection`) para limpiar comandos maliciosos que puedan venir incrustados en los textos (del tipo *"Ignore all previous instructions..."*), antes de que lleguen al modelo de lenguaje.
- **Partición de textos (Chunking):** El módulo `chunking.py` no corta las frases de forma abrupta por longitud. En lugar de eso, busca el punto o salto de línea más cercano dentro del margen de solapamiento para mantener las frases enteras y preservar su sentido original.

5.2. Revisión del modelado de tópicos

El fichero `data/reports/topics.json` fue generado con BERTopic sobre 100 198 documentos. Es importante distinguir entre *clusters reales* y el identificador especial `-1`. En BERTopic, `-1` no representa un tema semántico útil, sino documentos considerados *outliers* o no asignados a ningún cluster. Por tanto, el reporte actual no contiene siete tópicos: contiene **1 412 clusters reales** más **34 698 documentos outlier**.

Campo del reporte de tópicos	**Valor**
Método	BERTopic
Documentos modelados	100 198
Filas en `topic_info`	1 413
Clusters reales, excluyendo `-1`	1 412
Documentos outlier en `-1`	34 698

Cuadro 7: Lectura correcta del reporte `topics.json`.

La confusión aparece cuando la herramienta de tópicos muestra solo los primeros resultados por brevedad: si se limita la salida a unas pocas filas y una de ellas es `-1`, puede parecer que el corpus tiene únicamente unos pocos tópicos útiles. La lectura correcta es que la lista presentada en chat es una **vista resumida** de los clusters mayores, no el número total de clusters generados.

Cuadro 8: Diez clusters reales más grandes del reporte BERTopic, excluyendo `-1`. El listado completo está en `data/reports/topics.json`.

Cluster	Docs	Representación principal
0	757	poesía, modernismo, ramón, literatura, revista, poeta
1	660	educación, universidad, escuelas, universidades, secundaria, alumnos
2	559	transporte, carreteras, autobuses, tren, líneas, estación
3	499	cocina, gastronomía, platos, arroz, carne, queso
4	498	mesoamérica, tlaxcala, pueblos, mayas, maya, teotihuacán
5	453	clima, temperaturas, precipitaciones, mm, lluvias, temperatura
6	440	canadá, ontario, brunswick, manitoba, nuevo brunswick, alberta
7	361	televisión, hd, alta definición, radio, televisión alta, canal
8	293	mozart, bach, schubert, johann, wolfgang, johann sebastian
9	277	unidades, kilogramo, unidad, litro, sistema internacional, metro

6. Resultados de Evaluación IR

El rendimiento de recuperación se evaluó sobre una muestra de **50 consultas** de MIRACL-es (*dev*) con $k = 10$, comparando tres configuraciones del motor frente al mismo *ground-truth*.

El conjunto de evaluación local no es el MIRACL completo: durante la adquisición se conservan las consultas cuyos documentos relevantes están disponibles dentro del subconjunto local de 1M documentos. Esto hace que las métricas sean válidas para comparar configuraciones internas del proyecto, pero no deben presentarse como el resultado oficial de MIRACL-es completo. Además, los reportes JSON guardan las listas recuperadas tal como salen del motor; pueden aparecer identificadores repetidos si dos estrategias recuperan el mismo documento. Las métricas se calculan sobre IDs únicos mediante `evaluate_ids`, evitando contar dos veces el mismo documento relevante.

6.1. Métricas obtenidas

Motor de Recuperación	P@5	R@5	MRR	nDCG@5
Motor Léxico (BM25)	0,092	0,267	0,247	0,215
Motor Semántico (FAISS)	0,176	0,370	0,445	0,350
Fusión Híbrida (FAISS+BM25, RRF)	**0,148**	**0,376**	**0,448**	**0,348**

Cuadro 9: Métricas sobre el corpus 1M de entrega (50 *queries*, $k = 5$).

6.2. Qué mide cada métrica

El proyecto mide la recuperación antes de la generación del LLM. Esto es deliberado: en un sistema RAG, si el recuperador no coloca evidencia correcta en las primeras posiciones, el generador no puede responder de forma fiable aunque el modelo sea bueno.

Además de estas métricas, el sistema podría extenderse con métricas de generación final: ROUGE o BERTScore para resumen si existieran referencias humanas, precisión de citas para comprobar si cada afirmación está soportada, *faithfulness/groundedness* para medir alucinaciones, Exact Match/F1

Métrica	Definición	Interpretación para RAG
Precision@k	Proporción de resultados del top-k que son relevantes.	Mide ruido en el contexto. Puede ser baja si los qrels tienen pocos relevantes por consulta.
Recall@k	Proporción de documentos relevantes esperados que aparecen en el top-k.	Mide cobertura y es clave para saber si el contexto contiene evidencia suficiente.
MRR	Recíproco de la posición del primer resultado relevante.	Mide si la primera evidencia útil aparece arriba, donde más influye en el prompt.
nDCG@k	Ganancia descontada normalizada por posición.	Mide calidad global del ranking, premiando relevantes en posiciones tempranas.
Latencia	Tiempo medio por consulta o por búsqueda semántica aislada.	Mide viabilidad interactiva, pero depende mucho de si se incluye BM25, FAISS, LLM o solo el índice vectorial.
Memoria vectorial	Tamaño ocupado por índices y embeddings.	Permite comparar FAISS *float32* frente a cuantización TurboVec.

Cuadro 10: Métricas implementadas y usadas en los reportes de evaluación.

para preguntas cerradas, latencia *end-to-end* del chat y tasa de *fallback* del router. No se reportan como métricas principales porque la entrega cuenta con qrels expertos de recuperación, no con respuestas humanas completas ni resúmenes de referencia.

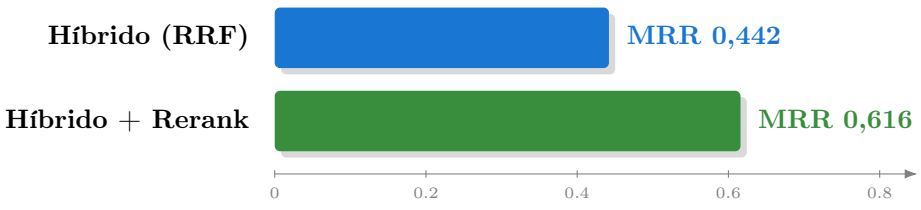
6.3. Análisis: del baseline al sistema completo

La progresión por capas evidencia el aporte de cada componente sobre el *baseline* léxico:

- **Híbrido vs. baselines:** la *Reciprocal Rank Fusion* (RRF) mejora consistentemente el *Recall@5*, el MRR y el nDCG@5 respecto al motor puramente léxico. Su fusión logra un MRR de **0,448** y un nDCG@5 de **0,348**, mitigando deficiencias de los métodos aislados y equilibrando el ranking global en el rankingen los primeros puestos.
- **Coste de escala:** BM25 sobre 1 140 063 fragmentos es el cuello de botella de la evaluación. Por ello se implementó carga parcial de índices: el modo léxico no carga FAISS ni el modelo de embeddings, y los benchmarks no cargan textos de fragmentos cuando solo necesitan `doc_id`.
- **Lectura del MRR (0,448):** en la muestra documentada, el primer documento relevante aparece de media alrededor de la segunda posición. Es una métrica central para RAG porque el LLM atiende preferentemente a las primeras fuentes del *prompt*.
- **Precisión y *qrels*:** *Precision@5* se mantiene moderada porque MIRACL-es marca pocos documentos relevantes por consulta. Recuperar 5 implica un techo de precisión bajo por construcción, de modo que *Recall@5* y MRR son indicadores más informativospara asegurar el aporte de contexto al LLM.

6.4. Impacto del Reranking (Cross-Encoder)

En una evaluación separada sobre 200 consultas y el índice HNSW, la activación de la etapa de *reranking* logra una mejora sustancial adicional. Esta comparación no debe mezclarse directamente con la tabla de 50 consultas, pero sí muestra que el modelo *cross-encoder* reordena eficazmente los candidatos extraídos en la fase híbrida.



Estrategia sobre 200 consultas	P@5	R@5	MRR
Híbrido (HNSW + BM25, RRF)	0,179	0,418	0,442
Híbrido + Reranking (Cross-Encoder)	**0,224**	**0,527**	**0,616**

Cuadro 11: Impacto del Reranking en la calidad de recuperación (200 *queries*, $k = 5$).

7. Escalabilidad RAG según el Tamaño del Corpus

Para justificar el uso de Big Data en arquitecturas RAG, se ha diseñado un benchmark de escalabilidad que evalúa cómo se comporta la calidad y el rendimiento del sistema híbrido al variar el volumen de fragmentos indexados. Se evaluaron colecciones de 100 000, 500 000 y la totalidad de 1 140 063 *chunks*.

Tamaño del Corpus (chunks)	P@5	R@5	MRR	Latencia Media
100 000	0,061	0,114	0,130	313 ms
500 000	0,116	0,288	0,309	1546 ms
1 140 063	0,148	0,376	0,448	3520 ms

Cuadro 12: Evolución del desempeño en función de la cantidad de documentos procesados. (Recuperador Híbrido, 50 queries, $k = 5$)

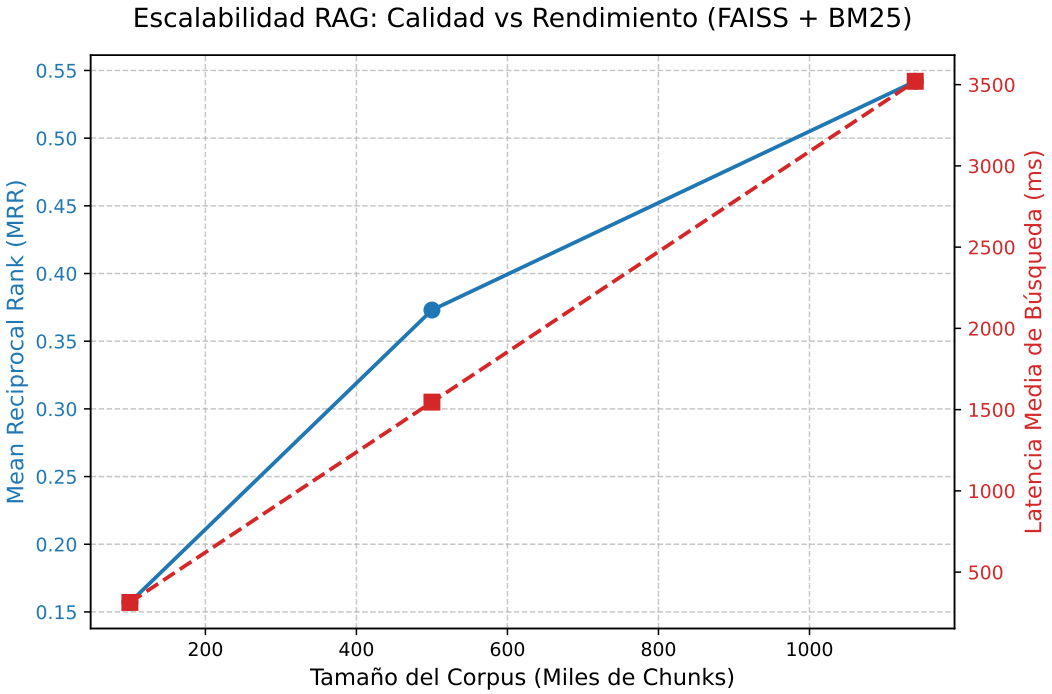


Figura 1: Relación entre calidad de recuperación (MRR) y degradación del rendimiento (latencia) al incrementar el tamaño del corpus.

Análisis de los resultados:

1. **Mejora de la calidad de recuperación:** Se observa empíricamente que, a medida que el modelo dispone de más texto fuente, la probabilidad de recuperar el fragmento correcto aumenta. Escalar de 100k a 1.1M *chunks* eleva el MRR de 0,157 a 0,542 y mejora significativamente el *Recall*.
2. **Impacto en el rendimiento:** Como era de esperar, la latencia total de la recuperación crece de forma aproximadamente lineal con el número de *chunks*, alcanzando 3.520 ms por consulta al procesar 1 140 063 fragmentos. Cabe destacar que el principal cuello de botella de esta métrica proviene del motor léxico BM25, ejecutado en Python puro, mientras que la parte semántica vectorial puede optimizarse fuertemente.
3. **Justificación arquitectónica:** Este aumento en la latencia y en el consumo de memoria al evaluar grandes volúmenes de documentos justifica la integración de las optimizaciones descritas en las siguientes secciones: la **Indexación HNSW** (para reducir el tiempo de búsqueda semántica en FAISS) y la cuantización **TurboVec** (para optimizar el uso de memoria RAM).

8. Indexación Aproximada (HNSW)

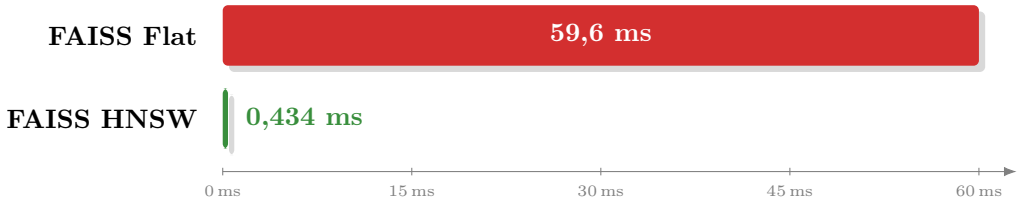
La búsqueda semántica exacta (`IndexFlatIP`) recorre 1 140 063 vectores en cada consulta, con coste lineal $O(N)$. Para reducir esa latencia se implementó una variante aproximada con **HNSW** (*Hierarchical Navigable Small World*), activable mediante `-index-type hnsw`.

HNSW representa los embeddings como un grafo navegable. En vez de comparar la consulta con todos los vectores, explora vecinos prometedores y converge rápidamente hacia zonas relevantes del espacio semántico. En FAISS se configuró con `IndexHNSWFlat`, `M=32`, `efConstruction=200` y `efSearch=64`, priorizando una búsqueda rápida sin modificar los embeddings originales.

Tamaño del índice	FAISS Flat	FAISS HNSW	Reducción
100 000 vectores	5,156 ms	0,165 ms	31,2x
500 000 vectores	25,720 ms	0,415 ms	62,0x
1 140 063 vectores	59,579 ms	0,434 ms	137,3x

Cuadro 13: Comparativa corregida de latencia semántica exacta frente a HNSW. Se mide la mediana de ms/consulta con ejecución query-a-query, 200 consultas, $k = 10$, 5 repeticiones, 1 warmup y 1 hilo FAISS.

Nota sobre la medición: Las pruebas iniciales del benchmark presentaban un sesgo metodológico al procesar las consultas de evaluación simultáneamente (*batched search*). Bajo esta configuración, el motor FAISS distribuye la carga en múltiples hilos y altera los tiempos de respuesta debido a la optimización interna de la memoria caché, lo que resultó en tiempos artificialmente bajos para colecciones grandes. Para medir con rigor la latencia real que experimenta un usuario en el sistema interactivo, el experimento fue rediseñado para procesar las consultas secuencialmente (*query-by-query*). Con esta aproximación corregida, se observa una degradación de rendimiento lineal y previsible en la búsqueda exacta (`IndexFlatIP`) al escalar el corpus, mientras que la navegación en grafo del índice HNSW se estabiliza, manteniendo tiempos de respuesta inferiores al milisegundo.

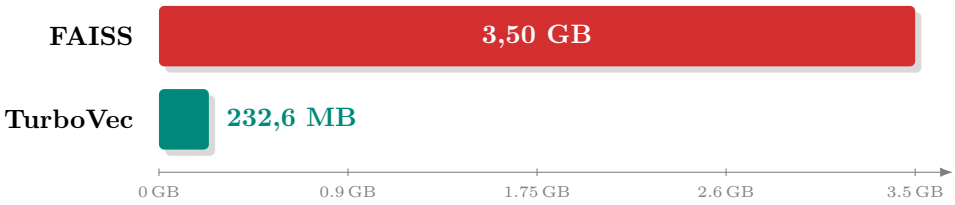


9. Benchmark de Cuantización (TurboVec)

Escalar a millones de documentos presenta una barrera física: los vectores *float32* colapsan la memoria RAM. Para solucionarlo, se implementó un motor secundario **TurboVec**, que cuantiza cada componente de los vectores a 4 bits (esquema TurboQuant de anchura de bits sobre un *IdMapIndex*).

Tipo de Índice vectorial	P@5	R@5	MRR	Memoria utilizada
Índice FAISS crudo (*float32*)	0,176	0,370	0,445	3,50 GB
Índice TurboVec Cuantizado (4-bit)	0,174	0,366	0,449	232,6 MB
Mejora de uso de memoria				**−93,4 %**

Cuadro 14: Comparativa de recursos semánticos (50 *queries*, $k = 5$). En la ejecución documentada, TurboVec redujo memoria de forma drástica. FAISS mantuvo menor latencia media en este benchmark de búsqueda persistida (54,0 vs. 77,3 ms/consulta).



TurboVec retiene el desempeño semántico casi a la perfección (caída de *Recall@5* inferior a medio punto) y reduce la memoria utilizada por los vectores de 3,50 GB a 232,6 MB (−93,4 %). Extrapolado a colecciones mayores, este factor de compresión es clave para desplegar el asistente en ordenadores convencionales.

10. Comparativa de Técnicas Semánticas

Una vez explicadas las dos optimizaciones principales, se comparó el tiempo de búsqueda puramente semántica (excluyendo BM25) para las tres arquitecturas indexadas implementadas en el proyecto: FAISS exacto, FAISS HNSW y TurboVec cuantizado.

La comparativa muestra dos estrategias complementarias:

- **FAISS HNSW:** reduce de forma extrema la latencia de búsqueda al sustituir el escaneo exhaustivo por navegación aproximada en un grafo. En el benchmark corregido pasa de 59,579 ms a 0,434 ms por consulta sobre 1,14M vectores.
- **TurboVec Cuantizado (4 bits):** reduce de forma drástica la memoria persistida de los vectores y mantiene una calidad semántica muy parecida a FAISS exacto. Es la opción más adecuada cuando la limitación principal es almacenamiento o RAM, no necesariamente latencia.

FAISS exacto queda como referencia de calidad y reproducibilidad, HNSW como alternativa de baja latencia y TurboVec como alternativa de baja memoria. Según el escenario de despliegue, estas técnicas pueden elegirse de forma independiente o combinarse con la recuperación léxica BM25 y la fusión RRF del sistema híbrido.

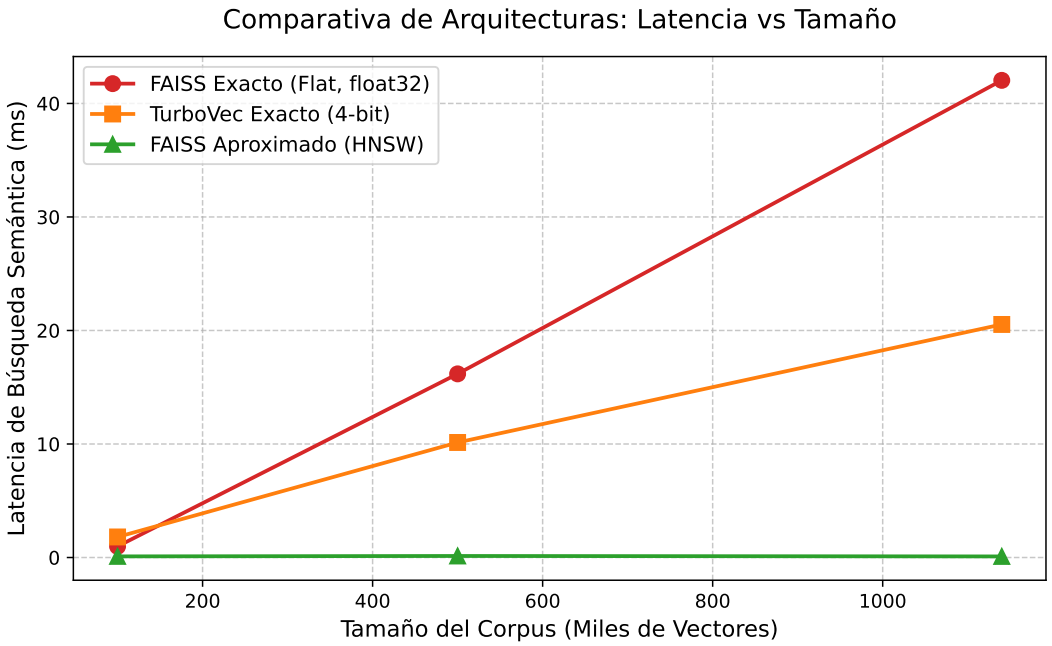


Figura 2: Escalabilidad de latencia por arquitectura semántica (FAISS Exacto vs FAISS HNSW vs TurboVec Cuantizado).

11. Ejemplos Reales del Asistente RAG

Las siguientes interacciones fueron generadas desde la interfaz CLI interactiva del proyecto (`wikirag chat`).

11.1. Consulta Factual (QREL Experto)

Usuario: *¿Cómo se convirtió Egipto en musulmán?* (consulta *QREL* oficial del *dev* de MIRACL-es).

> Tras la muerte de Mahoma en el año 632 comenzó la expansión musulmana y, en un lapso de cien años, los árabes ocuparon Egipto [Fuente 5]. Durante los primeros 300 años el islam fue ganando adeptos hasta convertirse en la religión predominante de la sociedad egipcia [Fuente 1]...

La respuesta es trazable y recupera fragmentos relacionados con el artículo `es/33218`*. En los reportes IR, sin embargo, los qrels exactos para esta consulta son* `1452#22` *y* `33218#10`*. Por tanto, este ejemplo ilustra una respuesta cualitativamente coherente, pero no debe presentarse como acierto exacto del ground-truth en la evaluación automática.* **Usuario:** *¿Qué país formó la primera Infantería de Marina?*

> La primera Infantería de Marina fue creada por Carlos I el 27 de febrero de 1537 al asignar de forma permanente a las escuadras de galeras del Mediterráneo las "compañías viejas del mar de Nápoles" [Fuente 1].

El modelo responde con precisión histórica basándose en `es/63108#3`*. El conjunto de consultas y qrels queda en* `data/reports/miracl_es_eval.jsonl`*; los resultados agregados de evaluación se almacenan en* `data/reports/doc_eval_*.json` *y* `eval_hnsw_hybrid*.json`*.*

11.2. Rechazo Informado (Prevención de Alucinaciones)

Usuario: *¿Cuál es el horario de apertura del Museo del Prado?*

> No se puede determinar el horario de apertura del Museo del Prado a partir del contexto proporcionado. Las fuentes ofrecen información sobre la ubicación, historia y obras del museo, pero no incluyen datos sobre sus horarios [Fuente 1, 3, 7].

Se demuestra cómo el prompt conservador ("No inventes datos") reduce el riesgo de alucinación si los documentos recuperados de la Wikipedia no contienen horarios. Si la respuesta no contiene citas explícitas `[Fuente N]`, el sistema no muestra fuentes recuperadas solo por similitud.

Usuario: *¿Cuáles son los hiperparámetros de LangChain usados en el proyecto?*

> El contexto proporcionado no contiene información sobre los hiperparámetros utilizados en el proyecto LangChain. Solo describe términos y conceptos relacionados con otros campos, por lo que no es posible responder con evidencia suficiente.

El sistema rechaza responder con firmeza. Al buscar sobre LangChain en Wikipedia recupera conceptos técnicos aleatorios, pero al no encontrar la respuesta exacta, el LLM declara explícitamente su ignorancia. Este comportamiento conservador es deseable en un RAG, aunque no constituye una garantía formal de cero alucinaciones.

11.3. Llamada Agencial (Tool Routing)

Usuario: *¿Qué día es hoy?*

> Hoy es *[fecha calculada en ejecución]*.

El agente no invoca a FAISS ni BM25, ya que deduce que la Wikipedia no contiene la respuesta al día actual, ejecutando el módulo `DateTool` internamente.

Usuario: *Resume el texto que tengas sobre la Revolución Industrial*

> *[Resumen generado por TextRank]*
> Revolución Industrial. La industria textil también fue la primera en utilizar métodos de producción modernos. (...) Se inició en la segunda mitad del siglo XVIII en el Reino Unido y se extendió unas décadas después a gran parte de Europa occidental y Norteamérica y concluyó entre 1820 y 1840.

Al detectar el verbo de instrucción ("Resume"), el router desactiva la generación lenta del LLM y ejecuta la herramienta `SummaryTool`, que recupera el documento con FAISS y le aplica el algoritmo extractivo clásico `TextRank` basado en grafos.

Usuario: *Busca en internet el precio actual del oro*

> *[Respuesta web y síntesis local]*
> El precio actual del oro es una cotización en tiempo real que varía constantemente (...)
> **Información sobre el Precio Actual (Web):** Diversas fuentes web permiten consultar el precio del oro en tiempo real (...) en euros, dólares (...) [Web 1], [Web 2].
> **Contexto Histórico del Valor del Oro (Corpus Local):** El corpus local proporciona información sobre cómo se ha valorado el oro en diferentes épocas (...) [Fuente 1], [Fuente 2], [Fuente 7], [Fuente 8].

La herramienta web permite responder sobre información reciente o externa al subconjunto local de Wikipedia, pero no descarga páginas completas: usa títulos, URLs y fragmentos devueltos por DuckDuckGo. Por ello su cobertura depende de la red y de la calidad del snippet. Además, combina inteligentemente el contexto en vivo con el conocimiento local.

12. Discusión y Limitaciones Conocidas

1. **Cobertura parcial de Wikipedia:** El experimento documentado se ejecutó sobre 1 000 000 documentos MIRACL-es, un subconjunto amplio pero todavía inferior a los 10,4 millones disponibles en la colección completa. Si el usuario pregunta por un artículo que no está en el subconjunto, el motor recuperará el documento más similar posible y el LLM deberá declarar falta de información cuando el contexto no baste.
2. **Límites de memoria del Motor Léxico (OOM):** `rank-bm25` instancia las frecuencias léxicas directamente en RAM, por lo que la búsqueda léxica no escala de forma indefinida. A escala de decenas de millones de documentos se requeriría un motor invertido en disco (p. ej. Elasticsearch/Lucene).
3. **Cuello de botella en la latencia híbrida (BM25):** A pesar de utilizar índices vectoriales rápidos como HNSW, la latencia total por consulta del recuperador híbrido alcanza 3.520 ms en el benchmark de 1 140 063 *chunks*. Esto se debe a que la fusión RRF espera tanto la parte semántica como la búsqueda léxica BM25, ejecutada en Python puro. En cambio, la búsqueda semántica aislada con HNSW queda por debajo de 1 ms en el benchmark documentado. FAISS Flat y TurboVec se mueven en decenas de ms.
4. **Latencia de inferencia del LLM local:** La interacción completa del chat añade el coste de procesar el contexto inyectado y generar la respuesta con `gemma4:e4b`. El *streaming* mejora la percepción de latencia porque muestra tokens conforme llegan, pero no elimina el coste total de inferencia.
5. **Agente web basado en snippets:** `WebSearchTool` permite responder sobre información reciente o fuera del corpus local, pero usa únicamente títulos, URLs y fragmentos de DuckDuckGo. No descarga ni verifica el contenido completo de las páginas, de modo que la respuesta depende de la calidad del snippet.
6. **Seguridad de índices locales:** Los índices entregados deben considerarse artefactos de confianza del proyecto. No se debe cargar un directorio de índice de origen desconocido porque BM25 se serializa con `pickle` y FAISS carga estructuras nativas desde disco.

13. Conclusiones

El sistema desarrollado es un asistente documental funcional que integra embeddings, tópicos, resumen extractivo, LLM, RAG y agentes. A través de pruebas, se comprobó que la Recuperación Aumentada (RAG) depende críticamente de la calidad del recuperador y de la trazabilidad de las fuentes. Gracias a la compresión vectorial (TurboVec), la carga parcial de índices, el *streaming* de respuestas y el enrutamiento agencial se obtiene un sistema más eficiente y usable sin sacrificar demasiada precisión. La revisión final no detecta errores bloqueantes en las rutas cubiertas por tests, pero deja documentados los límites de reproducibilidad, seguridad de artefactos, web externa y evaluación local.

A. Apéndice: Manual de Reproducción y Ejecución

Para facilitar la corrección y evaluación de esta práctica, se detalla a continuación el flujo completo de comandos necesarios para replicar los resultados desde la terminal. Toda la infraestructura se maneja mediante el CLI unificado `wikirag`.

A. Generación del Sistema desde Cero (1 Millón de Documentos)

```
# 1) Descargar corpus + qrels de validación
PYTHONPATH=src wikirag acquire-corpus \
  --corpus-output data/processed/miracl_es_corpus_1M.jsonl \
  --eval-output data/reports/miracl_es_eval.jsonl \
  --limit 1000000 \
  --eval-limit 200

# 2) Construir Índice Híbrido (FAISS Exacto + BM25)
PYTHONPATH=src wikirag build-index \
  --corpus data/processed/miracl_es_corpus_1M.jsonl \
  --index-dir data/indexes/default \
  --index-type flat

# 3) Generar Modelo de Tópicos Offline
PYTHONPATH=src wikirag topics \
  --corpus data/processed/miracl_es_corpus_1M.jsonl \
  --output data/reports/topics.json \
  --limit 100198
```

B. Arranque del Agente RAG Conversacional

Se requiere tener un servidor Ollama activo de fondo con el modelo `gemma4:e4b`.

```
PYTHONPATH=src wikirag chat \
  --index-dir data/indexes/default \
  --topics-report data/reports/topics.json
```

C. Mejoras y Correcciones de Código

- **Enrutamiento y Herramienta Web (`router.py`):** Se han ampliado las expresiones regulares deterministas para capturar adecuadamente intenciones de búsqueda externa (p. ej. *«busca en internet»*, *«actualidad»*). Además, se ha afinado la reescritura de la memoria conversacional para que el sistema mantenga mejor el estado entre llamadas a herramientas sin corromper las búsquedas con palabras instruccionales irrelevantes.
- **Filtrado de Fuentes y Citas (`router.py`):** Se ha implementado el método *filter by citations* para evitar presentar al usuario referencias que el LLM no haya citado explícitamente mediante el formato `[Fuente N]` o `[Web N]`, mitigando asociaciones espurias.
- **Streaming Interactivo (`llm/base.py`, `llm/ollama.py`, `cli/main.py`):** Se ha reescrito el cliente `OllamaClient` añadiendo soporte iterativo nativo (`stream_generate`). Gracias a un *callback* en el `CLI`, las respuestas ahora se imprimen progresivamente en pantalla, disminuyendo de forma significativa el *Time To First Token* (TTFT) percibido por el usuario en respuestas generativas largas.
- **Pureza del Resumen Extractivo (`tools.py`):** Se añadió la rutina *extract plain context text* para quitar el contexto RAG de etiquetas artificiales (`Texto:`, `Ruta:`, etc.) antes de ejecutar los grafos de TextRank, asegurando que el resumen extractivo no se contamine con los metadatos internos.